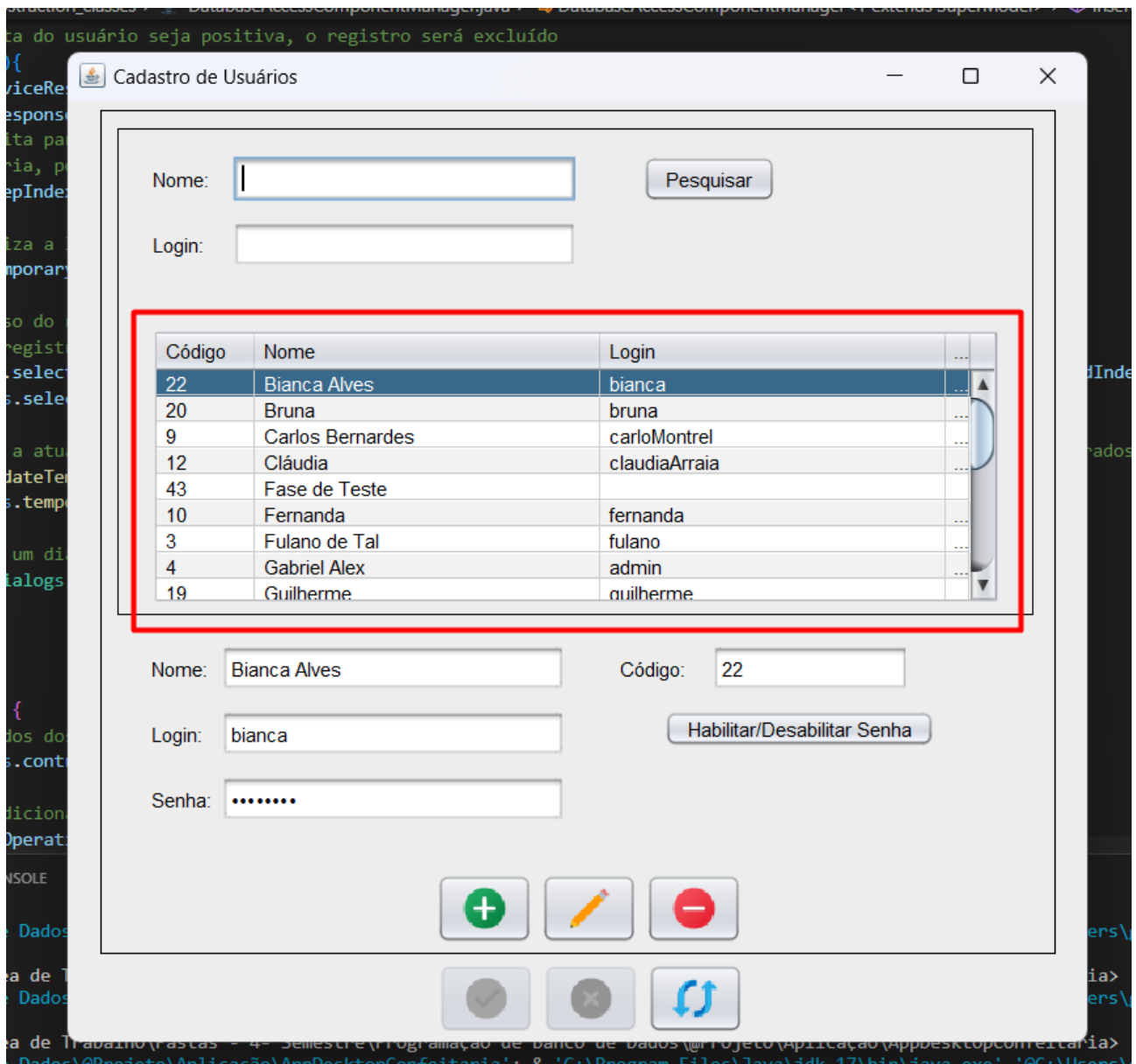


CRUD Usuário

Select

Ao seleccionar os dados no banco, quem é responsável por exibi-los é a JTable(tabela do java swing):



Ela possui campos para que se possa filtrar a pesquisa. Porém, por padrão, a pesquisa inicial vem como total, sem nenhuma condição. É o mesmo método usado para fazer a pesquisa parcial, basta passar os parâmetros como nulos que a classe UsuarioDAO cuidará do resto, veja a seguir:

```
public List<Usuario> partialSearch(String nome, String login) {
    PreparedStatement statement = null;
    ResultSet resultSet = null;

    /*Verifica se os parâmetros foram fornecidos e se não são vazios, ou seja,
    confere se o usuário deseja realizar a pesquisa por nome e/ou login*/
    boolean selectByName = nome != null && !nome.isEmpty();
    boolean selectByLogin = login != null && !login.isEmpty();

    //Cria a query
    String sql =
        "SELECT *"+
        "FROM usuario " +
        "WHERE 1 = 1" +
        (selectByName ? "AND unaccent(nome) ILIKE unaccent(?)" : "") + //Se o usuário deseja pesquisar por nome, adiciona a condição à query
        (selectByLogin ? "AND unaccent(login) ILIKE unaccent(?)" : "") + //Se o usuário deseja pesquisar por login, adiciona a condição à query
        "ORDER BY nome, login";

    /*Define o padrão de pesquisa em relação aos parâmetros fornecidos pelo usuário*/
    nome = "%" + nome + "%";
    login = "%" + login + "%";

    //Define uma variável para armazenar o índice do parâmetro a ser configurado
    int paramIndexCount = 1;
    try{
        //Define o PreparedStatement com o SQL
        statement = DBConnection.getConnection().prepareStatement(sql);

        //Configura os parâmetros necessários
        if(selectByName) {
            statement.setString(paramIndexCount, nome);
            paramIndexCount++; //Incrementa o índice do parâmetro para que o próximo seja configurado corretamente
        }
        if(selectByLogin) {
            statement.setString(paramIndexCount, login);
        }

        //Armazena o SQL para haver referência da última consulta realizada pelo DAO
        this.lastSqlPartialSearch = statement.toString();

        //Obtém o ResultSet executando a query
        resultSet = statement.executeQuery();

        return this.resultSetToList(resultSet);
    } catch (SQLException e) {
        e.printStackTrace();
        return null;
    } finally {
        //Fecha a conexão com o banco de dados e os recursos criados a partir dela
        DBConnection.closeConnection(statement, resultSet);
    }
}
```

Por tanto, antes de exibir a tela, a View solicitará uma pesquisa total a Service que por sua vez acionará o método da DAO mostrado acima, passando parâmetros nulos. A DAO retornará o resultado para a Service, que por sua vez retornará para a View.

Para configurar a JTable para exibir os dados obtidos do banco, o seguinte código é aplicado:

```
private void displayDataInTable(List<T> tList) {
    //Limpando a tabela
    this.tableModel.setNumRows(rowCount:0);

    if(tList != null && tList.size() > 0) {
        //Adicionando os dados na tabela
        for(T tObject : tList) {
            try {
                this.tableModel.addRow(this.controller.modelToTableRow(tObject));
            } catch (Throwable e) {
                e.printStackTrace();
            }
        }
    }
}
```

Porém, esse método é de uso geral, para qualquer tabela da aplicação, cada tela tem que implementar um mapeamento do modelo do objeto para o formato da sua própria linha da tabela. A do usuário é o seguinte:

```
@Override
public Object[] modelToTableRow(Usuario usuario){
    return new Object[]{
        usuario.getID(),
        usuario.getNome(),
        usuario.getLogin(),
        usuario.getSenha()
    };
}
```

Toda a mesma lógica apresentada também é usada nas pesquisas parciais, com as únicas diferenças de que:

- O botão pesquisar da View aciona a pesquisa.
- Ao ser acionado por clique, os dados dos campos de pesquisa são capturados e repassados como argumentos para a Service, que por sua vez os usará para fazer a pesquisa parcial por meio do método da DAO descrito anteriormente.

Insert

Quando o botão de cadastro de registro é acionado as seguintes operações ocorrem na tela:

- Uma linha é adicionada na tabela indicando que um novo registro está sendo inserido.
- A tabela é desativada(para que o usuário não troque o registro selecionado no meio da edição).
- Os botões de edição(insert, update e delete) são desativados.
- Os botões de confirmação de edição(post e cancel) e o de atualização(refresh) são ativados.

The screenshot shows a web application window titled "Cadastro de Usuários". It contains a search section with "Nome:" and "Login:" input fields and a "Pesquisar" button. Below is a table of users. The table has columns "Código", "Nome", and "Login". The row for "José" (Código 11) is highlighted in blue. Below the table are input fields for "Nome:", "Login:", and "Senha:", each with a red error message "Campo obrigatório !!!". There is also a "Código:" input field and a "Habilitar/Desabilitar Senha" button. At the bottom, there are three buttons: a green checkmark, a red X, and a blue refresh icon. The "Insere" button, which is a grey circle with a plus sign, is highlighted with a red box.

Código	Nome	Login
50	João Alves	joaoAlves21
48	João Mario	joaomario
70	João Paulo	jp007400
11	José	jose
69	Leia Oliveira	leiaOLiviera
14	Leticia	leticia
15	Lucas	lucas
13	Luiz Antonio	luizAMarques

Caso o usuário cancele a inserção, todos os dados inseridos nos campos serão limpos e a linha que havia sido adicionado na tabela será apagada. Caso ele deseje inserir os dados, apertando o botão salvar(post), ocorrem as seguintes operações, veja as imagens abaixo:

The interface displays a table of users and a form to add a new user. The table has columns for Código, Nome, and Login. The form includes fields for Nome, Código, Login, and Senha, along with a 'Habilitar/Desabilitar Senha' button. Below the form are three buttons: a green checkmark (highlighted with a red box), a red X, and a refresh icon. A 'Salvar' label is positioned below the green checkmark button.

Código	Nome	Login
50	João Alves	joaoAlves21
48	João Mario	joamario
70	João Paulo	jp007400
11	José	jose
69	Leia Oliveira	leiaOLiviera
14	Leticia	leticia
15	Lucas	lucas
13	Luiz Antonio	luizAMarques

Nome: Código:

Login:

Senha:

Salvar

The interface shows the same table and form, but the new user 'Ana Maria Silva' with code '220' is now listed in the table. The 'Salvar' button is no longer highlighted.

Código	Nome	Login
50	João Alves	joaoAlves21
48	João Mario	joamario
70	João Paulo	jp007400
11	José	jose
220	Ana Maria Silva	anamaria
69	Leia Oliveira	leiaOLiviera
14	Leticia	leticia
15	Lucas	lucas
13	Luiz Antonio	luizAMarques

Nome: Código:

Login:

Senha:

Em código, quando o salvar é clicado, os dados dos campos são transformados em um objeto da classe usuário e o seguinte método da Service é acionado:

```
@Override
public boolean insert(Usuario usuario) {
    //Valida os dados do usuário
    if(this.validateDataInsert(usuario)) {
        /*Caso os dados sejam válidos, solicita ao DAO a inserção
        do usuário no banco de dados já retornando o resultado*/
        return this.usuarioDAO.insert(usuario);
    } else {
        return false;
    }
}
```

Na DAO, ocorre o seguinte:

```
public boolean insert(Usuario usuario) {
    PreparedStatement statement = null;
    try {
        //Verifica se o ID do usuário foi fornecido
        if(usuario.getID() == null) {
            //Se não foi, realiza a inserção sem o ID
            this.insertWithoutId(usuario, statement);
        } else {
            //Se foi, realiza a inserção com o ID
            this.insertWithId(usuario, statement);
        }
        //Se a inserção foi realizada com sucesso, retorna true
        return true;
    } catch (SQLException e) {
        e.printStackTrace();
        //Se a inserção não foi realizada com sucesso, retorna false
        return false;
    }
}

/*Método para inserir um usuário sem o ID. O método é privado pois as solicitações de inserção
devem ser feitas através do método insert(Usuario usuario) para que o ID seja verificado por ele*/
private void insertWithoutId(Usuario usuario, PreparedStatement statement) throws SQLException {
    //Cria a query
    String sql = "INSERT INTO usuario(nome, login, senha) " +
        "VALUES(?, ?, ?) RETURNING id_usuario";

    //Define o PreparedStatement com o SQL, em seguida, configura os parâmetros necessários
    statement = DBConnection.getConnection().prepareStatement(sql);
    statement.setString(parameterIndex:1, usuario.getNome());
    statement.setString(parameterIndex:2, usuario.getLogin());
    statement.setString(parameterIndex:3, usuario.getSenha());

    //Executa a query e obtém o ResultSet(que deverá conter o ID do usuário retornado pela inserção)
    ResultSet resultSet = statement.executeQuery();
    if(resultSet.next()) {
        //Se a inserção foi realizada com sucesso, define o ID do usuário
        usuario.setID(resultSet.getLong(columnLabel:"id_usuario"));
    }
    //Fecha a conexão com o banco de dados e os recursos criados a partir dela
    DBConnection.closeConnection(statement);
}
```

```

/*Método para inserir um usuário com o ID. O método é privado pois as solicitações de inserção
devem ser feitas através do método insert(Usuario usuario) para que o ID seja verificado por ele*/
private void insertWithId(Usuario usuario, PreparedStatement statement) throws SQLException {
    //Cria a query
    String sql = "INSERT INTO usuario(id_usuario, nome, login, senha) " +
        "VALUES(?, ?, ?, ?)";

    //Define o PreparedStatement com o SQL, em seguida, configura os parâmetros necessários
    statement = DBConnection.getConnection().prepareStatement(sql);
    statement.setLong(parameterIndex:1, usuario.getID());
    statement.setString(parameterIndex:2, usuario.getNome());
    statement.setString(parameterIndex:3, usuario.getLogin());
    statement.setString(parameterIndex:4, usuario.getSenha());

    //Executa a query
    statement.executeUpdate();

    //Fecha a conexão com o banco de dados e os recursos criados a partir dela
    DBConnection.closeConnection(statement);
}

```

Update

No update, basta que o usuário aperte o botão de edição(o lápis amarelo) ou que alterar o valor de algum dos campos que o sistema saberá que o registro está sendo editado. As mesmas operações visuais do insert ocorrem: Uma linha é adicionada na tabela, a tabela é desativada, os botões de edição são desativados e os botões de confirmação de edição e o de atualização são ativados.

Segue um exemplo completo:

The screenshot displays a user management interface. At the top, there is a table with columns 'Código', 'Nome', and 'Login'. The table contains several rows of user data. The row with 'Código' 22 and 'Nome' 'Bianca Alves' is highlighted with a red rectangle. Below the table, there is a form for editing a user. The form has fields for 'Nome', 'Login', 'Senha', and 'Código'. The 'Nome' field contains 'Bianca Alves Amorim', the 'Login' field contains 'biancaAmorim', and the 'Código' field contains '22'. These fields are also highlighted with a red rectangle. To the right of the 'Código' field is a button labeled 'Habilitar/Desabilitar Senha'. At the bottom of the form, there are three buttons: a green checkmark, a red X, and a blue refresh icon. The bottom of the image shows a partial view of a file explorer window with the path 'Trabalho\trabalho - 4º Semestre\Programação de Banco de Dados\Projeto\Aplicação\appdesktop\conexao.java'.

Código	Nome	Login
220	Ana Maria Silva	anamaria
22	Bianca Alves	bianca
20	Bruna	bruna
9	Carlos Bernardes	carloMontrel
12	Cláudia	claudiaArraia
43	Fase de Teste	
10	Fernanda	fernanda
3	Fulano de Tal	fulano
4	Gabriel Alex	admin

Nome: Código:

Login:

Senha:

Buttons: +, ✎, -

Buttons: ✓, ✗, ↺

Trabalho\trabalho - 4º Semestre\Programação de Banco de Dados\Projeto\Aplicação\appdesktop\conexao.java

Se o salvar for clicado:

The screenshot shows a user management interface. At the top, there is a table with columns 'Código', 'Nome', and 'Login'. The table contains several rows of user data. The row with 'Código' 22, 'Nome' 'Bianca Alves Amorim', and 'Login' 'biancaAmorim' is highlighted with a red box. Below the table, there is a form for updating a user. The form has three input fields: 'Nome' (containing 'Bianca Alves Amorim'), 'Código' (containing '22'), and 'Login' (containing 'biancaAmorim'). The 'Nome' and 'Login' fields are grouped together with a red box. To the right of the 'Código' field is a button labeled 'Habilitar/Desabilitar Senha'. Below the input fields is a 'Senha' field with a masked password '.....'. At the bottom of the form, there are three buttons: a green '+' button, a yellow pencil icon button, and a red '-' button. Below these buttons are three more buttons: a grey checkmark button, a grey 'X' button, and a blue circular arrow button.

Código	Nome	Login
220	Ana Maria Silva	anamaria
22	Bianca Alves Amorim	biancaAmorim
20	Bruna	bruna
9	Carlos Bernardes	carloMontrel
12	Cláudia	claudiaArraia
43	Fase de Teste	
10	Fernanda	fernanda
3	Fulano de Tal	fulano
4	Gabriel Alex	admin

Nome:

Código:

Login:

Senha:

Habilitar/Desabilitar Senha

+ ✎ -

✓ ✕ ↺

Em código, a lógica é parecida com a operação do insert. Os dados dos campos são transformados em um objeto da classe usuário, o objeto do usuário contendo todos os dados originais também é capturado e o seguinte método da Service é acionado:

```
@Override
public boolean update(Usuario usuario, Usuario usuarioOriginal) {
    //Valida os dados do usuário
    if(this.validateDataUpdate(usuario, usuarioOriginal)) {
        /*Caso os dados sejam válidos, solicita ao DAO a atualização
        do usuário no banco de dados já retornando o resultado*/
        return this.usuarioDAO.update(usuario, usuarioOriginal.getID());
    } else {
        return false;
    }
}
```


Na DAO, ocorre o seguinte:

```
public boolean update(Usuario usuario, Long originalID) {
    if(originalID == null) return false;
    PreparedStatement statement = null;

    //Cria a query
    String sql = "UPDATE usuario " +
        "SET id_usuario = ?, nome = ?, login = ?, senha = ? " +
        "WHERE id_usuario = ?";

    try {
        //Define o PreparedStatement com o SQL, em seguida, configura os parâmetros necessários
        statement = DBConnection.getConnection().prepareStatement(sql);
        statement.setLong(parameterIndex:1, usuario.getID());
        statement.setString(parameterIndex:2, usuario.getNome());
        statement.setString(parameterIndex:3, usuario.getLogin());
        statement.setString(parameterIndex:4, usuario.getSenha());
        statement.setLong(parameterIndex:5, originalID);

        //Executa a query
        statement.executeUpdate();

        //Se a atualização foi realizada com sucesso, retorna true
        return true;
    } catch (SQLException e) {
        e.printStackTrace();
        //Se a atualização não foi realizada com sucesso, retorna false
        return false;
    } finally {
        //Fecha a conexão com o banco de dados e os recursos criados a partir dela
        DBConnection.closeConnection(statement);
    }
}
```

Delete

Por último, a operação delete. Para ela ser acionada, com um registro na tabela já selecionado, o botão de excluir deve ser clicado. Após isso, um dialog de confirmação de exclusão de registro será exibido na tela. Segue o exemplo:

The screenshot shows a user management interface. At the top, there is a table with three columns: 'Código', 'Nome', and 'Login'. The table contains several records, with the record having 'Código' 22 and 'Nome' 'Bianca Alves Amorim' selected. Below the table, there are input fields for 'Nome', 'Login', and 'Senha', along with a 'Habilitar/Desabilitar Senha' button. A modal dialog box titled 'Confirmação de exclusão' is displayed in the foreground. The dialog contains a question mark icon and the text 'Apagar o registro de código 22?'. At the bottom of the dialog are 'Ok' and 'Cancelar' buttons.

Código	Nome	Login
220	Ana Maria Silva	anamaria
22	Bianca Alves Amorim	biancaAmorim
20	Bruna	
9	Carlos Berr	
12	Cláudia	
43	Fase de Te	
10	Fernanda	
3	Fulano de T	
4	Gabriel Alex	

Nome: Bianca Alves Amorim Código: 22

Login: biancaAmorim

Senha:

Habilitar/Desabilitar Senha

Confirmação de exclusão

Apagar o registro de código 22?

Ok Cancelar

Caso o usuário clique em “Cancelar” nada acontecerá, caso aperte em “Ok” o registro será apagado do banco de dados e em seguida apagado da tabela.

Em código, caso o usuário tenha apertado em “Ok”, o método da Service mostrado abaixo é chamado:

```
@Override
public boolean delete(Usuario usuario) {
    //Solicita ao DAO a exclusão do usuário no banco de dados já retornando o resultado
    return this.usuarioDAO.delete(usuario.getID());
}
```

Na DAO:

```
public boolean delete(Long idUsuario) {
    if(idUsuario == null) return false;
    PreparedStatement statement = null;

    //Cria a query
    String sql = "DELETE FROM usuario " +
                 "WHERE id_usuario = ?";

    try {
        //Define o PreparedStatement com o SQL, em seguida, configura os parâmetros necessários
        statement = DBConnection.getConnection().prepareStatement(sql);
        statement.setLong(parameterIndex:1, idUsuario);

        //Executa a query
        statement.executeUpdate();

        //Se a exclusão foi realizada com sucesso, retorna true
        return true;
    } catch (SQLException e) {
        e.printStackTrace();
        //Se a exclusão não foi realizada com sucesso, retorna false
        return false;
    } finally {
        //Fecha a conexão com o banco de dados e os recursos criados a partir dela
        DBConnection.closeConnection(statement);
    }
}
```